

MLOps Assignment 3

Complete Project Documentation, Commands & Source Code

Ausaf Ahmad | MTech AI/ML | IIT Jodhpur | June 2026

Public Links — Access Everything Online

All services, dashboards, and resources used in this project:

Hugging Face

Model Page: <https://huggingface.co/knowledgator/gliclass-llama-1.3B-v1.0>

My Profile: <https://huggingface.co/ausafahmad>

Create Access Token: <https://huggingface.co/settings/tokens>

Weights & Biases (W&B)

Project Dashboard (mlops-assignment3): <https://wandb.ai/ausaf-ahmad-iitj3-indian-institute-of-technology-jodhpur/mlops-assignment3>

Run: gliclass-inference: <https://wandb.ai/ausaf-ahmad-iitj3-indian-institute-of-technology-jodhpur/mlops-assignment3/runs/kn6ofac0>

Run: kaggle-disaster-tweets-v3 (BEST): <https://wandb.ai/ausaf-ahmad-iitj3-indian-institute-of-technology-jodhpur/mlops-assignment3/runs/beguilibi>

Get W&B API Key: <https://wandb.ai/authorize>

Kaggle

My Profile: <https://www.kaggle.com/ausafiitj>

Competition: NLP Disaster Tweets: <https://www.kaggle.com/competitions/nlp-getting-started>

My Submissions: <https://www.kaggle.com/competitions/nlp-getting-started/submissions>

Leaderboard: <https://www.kaggle.com/competitions/nlp-getting-started/leaderboard>

Kaggle API Settings: <https://www.kaggle.com/settings>

Docker

Docker Hub: <https://hub.docker.com>

Docker Desktop Download: <https://www.docker.com/products/docker-desktop>

Docker Fedora Install Guide: <https://docs.docker.com/engine/install/fedora>

GitHub

GitHub: <https://github.com>

Create Personal Access Token: <https://github.com/settings/tokens>

Kaggle Submission Scores

Ref	Description	Date	Score	Status
53492993	GLiClass v1 threshold=0.3	2026-06-09 02:58	0.42966	COMPLETE
53502574	GLiClass v3 median threshold	2026-06-09 10:44	0.48605 ✓BEST	COMPLETE

Section 1: Environment Setup

1.1 Install Docker on Fedora WSL

```
sudo dnf remove -y docker docker-client docker-common
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo
https://download.docker.com/linux/fedora/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
sudo systemctl start docker && sudo systemctl enable docker
sudo usermod -aG docker $USER
newgrp docker
docker --version && docker run hello-world
```

1.2 Enable Docker WSL Integration

Docker Desktop → Settings → Resources → WSL Integration → Enable FedoraRemix → Apply & Restart

```
# Verify in WSL terminal
docker ps
```

1.3 Python Virtual Environment

⚠ venv MUST be in WSL home (~/) — NTFS F: drive blocks venv creation

```
sudo dnf install -y python3-pip python3-virtualenv
python3 -m venv ~/mlops_venv
source ~/mlops_venv/bin/activate
pip install ipykernel jupyter numpy pandas matplotlib huggingface_hub
transformers kaggle wandb torch gliclass flask
python -m ipykernel install --user --name=mlops_venv --display-name "Python
(MLOps-A3)"
jupyter kernelspec list
```

1.4 Daily Workflow

```
source ~/mlops_venv/bin/activate
cd "/mnt/f/MTechAI_ML_IITJ/Semester2/ML
Ops/Assignments/Assignment_3/work/MLOps_Assignment3V1"
code .
```

1.5 Windows → Linux Path Conversion

Windows	WSL/Linux
F:\MTechAI_ML_IITJ\...	/mnt/f/MTechAI_ML_IITJ/...

C:\Users\ausaf\	/mnt/c/Users/ausaf/
WSL Home (shortcut: ~)	\\wsl\$\FedoraRemix\home\ausafahmad
Folder with spaces: ML Ops	Wrap in quotes: "/mnt/f/ML Ops/..."

Section 2: Hugging Face Setup

2.1 Login

⚠ huggingface-cli is deprecated — use 'hf' command

```
pip install huggingface_hub
hf auth login          # paste token starting with hf_
hf auth whoami         # verify login
cat ~/.cache/huggingface/token # check stored token
```

2.2 Fix 401 Unauthorized Error

⚠ Old HF_TOKEN env var overrides hf auth login — must remove it first

```
# In Python:
import os
os.environ.pop('HF_TOKEN', None) # clear bad token
from huggingface_hub import login
login(token='hf_your_token_here')
```

2.3 Check Model Files

```
ls /mnt/f/models/gliclass-llama-1.3B
du -sh /mnt/f/models/gliclass-llama-1.3B
```

Section 3: Kaggle Setup

3.1 Create kaggle.json

```
mkdir -p ~/.kaggle
cat > ~/.kaggle/kaggle.json << 'EOF'
{"username":"ausafiitj","key":"YOUR_KAGGLE_KEY"}
EOF
chmod 600 ~/.kaggle/kaggle.json
cat ~/.kaggle/kaggle.json
```

3.2 Download Competition Data

[△ Must join competition at kaggle.com/competitions/nlp-getting-started](https://kaggle.com/competitions/nlp-getting-started) first

```
kaggle competitions download -c nlp-getting-started -p
/mnt/f/datasets/disaster-tweets
unzip /mnt/f/datasets/disaster-tweets/nlp-getting-started.zip -d
/mnt/f/datasets/disaster-tweets/
ls /mnt/f/datasets/disaster-tweets/
```

3.3 Submit & Check Score

```
kaggle competitions submit -c nlp-getting-started \
  -f /mnt/f/datasets/disaster-tweets/submission.csv \
  -m "GLiClass zero-shot v3 median threshold"

kaggle competitions submissions -c nlp-getting-started
```

Section 4: Weights & Biases Setup

4.1 Install & Login

[⚠](#) API key must be 40+ characters — get from wandb.ai/authorize

```
pip install wandb
wandb login --relogin
cat ~/.netrc # verify stored key
```

4.2 W&B Runs Created

Run Name	Run ID	Description
connection-test	kn6ofac0	First W&B test
gliclass-inference	kn6ofac0	Basic classification with score logging
kaggle-disaster-tweets	-	v1 all disaster (failed)
kaggle-disaster-tweets-v2	-	v2 all not disaster (failed)
kaggle-disaster-tweets-v3	beguilbi	v3 BEST — median threshold 0.48605

Section 5: Docker Configuration

5.1 Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

RUN pip install --no-cache-dir \
    gliclass \
    torch \
    huggingface_hub \
    wandb \
    pandas \
    flask

COPY run_model.py .

CMD ["python", "run_model.py"]
```

5.2 Build & Run Commands

```
# Build
docker build -t gliclass-model .
docker build --no-cache -t gliclass-model . # force fresh build

# Run basic
docker run --rm \
    -v "/mnt/f/models/gliclass-llama-1.3B:/models/gliclass-llama-1.3B:ro" \
    gliclass-model

# Run with W&B
docker run --rm \
    -v "/mnt/f/models/gliclass-llama-1.3B:/models/gliclass-llama-1.3B:ro" \
    -v ~/.netrc:/root/.netrc:ro \
    gliclass-model

# Run with Kaggle data + W&B
docker run --rm \
    -v "/mnt/f/models/gliclass-llama-1.3B:/models/gliclass-llama-1.3B:ro" \
    -v "/mnt/f/datasets/disaster-tweets:/data" \
    -v ~/.netrc:/root/.netrc:ro \
    gliclass-model

# Run Flask web UI at http://localhost:8080
docker run --rm -p 0.0.0.0:8080:8080 \
    -v "/mnt/f/models/gliclass-llama-1.3B:/models/gliclass-llama-1.3B:ro" \
    -v ~/.netrc:/root/.netrc:ro \
    gliclass-model
```


Section 6: All Python Files

Every Python file created during this project, in order. Dark background = Python source code.

□ `download_model.py`

Purpose: Download GLiClass model from Hugging Face to local F: drive

✓ Model downloaded to /mnt/f/models/gliclass-llama-1.3B (2.7 GB)

```
# download_model.py
# Run: python download_model.py

import os
from huggingface_hub import login, snapshot_download

# Remove any old bad token from environment
os.environ.pop('HF_TOKEN', None)

# Login with your Hugging Face token
login(token='hf_your_token_here')

# Download model to F: drive to save WSL home space
model_path = snapshot_download(
    repo_id='knowledgator/gliclass-llama-1.3B-v1.0',
    local_dir='/mnt/f/models/gliclass-llama-1.3B'
)

print('Model downloaded to:', model_path)

# List downloaded files
for f in os.listdir('/mnt/f/models/gliclass-llama-1.3B'):
    print(f)
```

□ test_connections.py

Purpose: Test all 3 service connections: HuggingFace, Kaggle, W&B

✓ All 3 services connected and verified

```
# test_connections.py
# Run: python test_connections.py

import os

# — 1. Hugging Face —
from huggingface_hub import whoami
os.environ.pop('HF_TOKEN', None)
user = whoami()
print(f'HF Connected: {user["name"]}')

# — 2. Kaggle —
import kaggle
kaggle.api.authenticate()
print('Kaggle Connected!')

# — 3. Weights & Biases —
import wandb
wandb.login()
run = wandb.init(project='mlops-assignment3', name='connection-test', mode='online')
print(f'W&B Connected! Run ID: {run.id}')
run.finish()

print('All services connected successfully!')
```

□ test_wandb.py

Purpose: Standalone W&B connection test

✓ W&B run created: ID kn6ofac0, project mlops-assignment3

```
# test_wandb.py
# Run: python3 /tmp/test_wandb.py

import wandb

wandb.login()
print('W&B Connected!')

run = wandb.init(project='mlops-assignment3', mode='online')
print('Run ID:', run.id)

run.finish()
print('Done!')
```

□ run_model.py v1 — Basic Classification

Purpose: First working version: load model and classify a single text

✓ Output: technology:0.3827 sports:0.3613 business:0.1716

⚠ Key fix: model.float() — converts BFloat16 weights to Float32 to fix RuntimeError

```
# run_model.py — Version 1: Basic classification
# Docker: docker run --rm -v model_path gliclass-model

from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer
import torch

model_path = '/models/gliclass-llama-1.3B'

print('Loading tokenizer...')
tokenizer = AutoTokenizer.from_pretrained(model_path)

print('Loading model...')
model = GLiClassModel.from_pretrained(model_path, torch_dtype=torch.float32)
model = model.float() # Fix: BFloat16 -> Float32

pipeline = ZeroShotClassificationPipeline(
    model, tokenizer,
    classification_type='multi-label',
    device='cpu'
)

# Test classification
text = 'Apple announces new iPhone with amazing features'
labels = ['technology', 'sports', 'politics', 'business']
result = pipeline(text, labels, threshold=0.1)

print('\n=== Classification Results ===')
for item in result:
    for pred in item:
        print(f"    {pred['label']}: {pred['score']:.4f}")
```

□ `run_model.py` v2 — With W&B Logging

Purpose: Log classification scores to W&B dashboard

✓ Scores logged: technology=0.3827, sports=0.3613, business=0.1716

```
# run_model.py — Version 2: W&B logging
# Docker: docker run --rm -v model -v ~/.netrc:/root/.netrc:ro gliclass-model

import wandb
import torch
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

# Initialise W&B run
run = wandb.init(
    project='mlops-assignment3',
    name='gliclass-inference',
    config={'model':'gliclass-llama-1.3B','threshold':0.1,'device':'cpu'}
)
print('W&B run ID:', run.id)

model_path = '/models/gliclass-llama-1.3B'
tokenizer = AutoTokenizer.from_pretrained(model_path)
model = GLiClassModel.from_pretrained(model_path, torch_dtype=torch.float32)
model = model.float()

pipeline = ZeroShotClassificationPipeline(
    model, tokenizer, classification_type='multi-label', device='cpu')

text = 'Apple announces new iPhone with amazing features'
labels = ['technology', 'sports', 'politics', 'business']
result = pipeline(text, labels, threshold=0.1)

print('\n=== Classification Results ===')
scores = {}
for item in result:
    for pred in item:
        print(f"    {pred['label']}: {pred['score']:.4f}")
        scores[pred['label']] = pred['score']

# Log scores to W&B
wandb.log(scores)
print('Logged to W&B:', scores)
run.finish()
```

□ run_model.py v3 — Flask Web Interface

Purpose: Interactive web UI at <http://localhost:8080> for zero-shot classification

✓ Web server running, browser UI accessible

```
# run_model.py — Version 3: Flask web app
# Docker: docker run --rm -p 0.0.0.0:8080:8080 -v model gliclass-model
# Then open: http://localhost:8080

import torch
import wandb
from flask import Flask, request, jsonify, render_template_string
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

app = Flask(__name__)

# Load model once at startup
tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model      = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                           torch_dtype=torch.float32)
model      = model.float()
pipeline   = ZeroShotClassificationPipeline(
    model, tokenizer, classification_type='multi-label', device='cpu')
print('Model ready!')

HTML = '''
<!DOCTYPE html><html><head><title>GLiClass Classifier</title>
<style>
  body{font-family:Arial;max-width:800px;margin:50px auto;padding:20px}
  textarea,input{width:100%;padding:10px;margin:10px 0;box-sizing:border-box}
  button{background:#4CAF50;color:white;padding:12px
24px;border:none;cursor:pointer;font-size:16px}
  .result{background:#f5f5f5;padding:15px;margin:10px 0;border-radius:5px}
  .bar{background:#4CAF50;height:22px;border-radius:3px;transition:width 0.3s}
</style></head><body>
  <h1>GLiClass Zero-Shot Classifier</h1>
  <textarea id='text' rows='4'
    placeholder='Enter text to classify...'>Apple announces new iPhone</textarea>
  <input id='labels' placeholder='Labels (comma separated)'
    value='technology,sports,politics,business,health'>
  <input id='threshold' type='number' step='0.05' min='0' max='1'
    value='0.1' placeholder='Threshold'>
  <button onclick='classify()'>Classify</button>
  <div id='results'></div>
<script>
  async function classify() {
    const text      = document.getElementById('text').value;
    const labels    = document.getElementById('labels').value
                      .split(',').map(l => l.trim());
    const threshold = parseFloat(document.getElementById('threshold').value);
    document.getElementById('results').innerHTML = '<p>Running...</p>';
    const res      = await fetch('/classify', {
      method:'POST', headers: {'Content-Type':'application/json'},
      body: JSON.stringify({text, labels, threshold})
    });
  };
```

```

const data = await res.json();
let html = '<h3>Results:</h3>';
data.results.forEach(r => {
  const pct = (r.score * 100).toFixed(1);
  html += `<div class='result'>
    <b>${r.label}</b>: ${pct}%
    <div style='background:#ddd;border-radius:3px'>
      <div class='bar' style='width:${pct}%'></div>
    </div></div>`;
});
document.getElementById('results').innerHTML = html;
}
</script>
</body></html>'''

@app.route('/')
def home():
    return render_template_string(HTML)

@app.route('/classify', methods=['POST'])
def classify():
    data = request.json
    text = data['text']
    labels = data['labels']
    threshold = data.get('threshold', 0.1)
    result = pipeline(text, labels, threshold=threshold)
    scores = {}
    for item in result:
        for pred in item:
            scores[pred['label']] = pred['score']

    try:
        wandb.log(scores)
    except:
        pass
    return jsonify({'results': [{ 'label':k, 'score':v} for k,v in scores.items()]})

if __name__ == '__main__':
    print('Starting web server at http://localhost:8080')
    app.run(host='0.0.0.0', port=8080)

```

❏ `run_model.py v4 - Kaggle v1 (threshold=0.3)`

Purpose: Classify all 3263 disaster tweets with fixed threshold 0.3

✓ **❌ FAILED** — all 3263 tweets predicted as disaster. Score: 0.42966

✗ **BUG:** threshold=0.3 too low — disaster scores mostly above 0.3 → all predictions = 1

```
# run_model.py - Version 4: Kaggle v1 (fixed threshold - FAILED)
# Kaggle score: 0.42966

import torch
import wandb
import pandas as pd
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

run = wandb.init(project='mlops-assignment3', name='kaggle-disaster-tweets')

tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model      = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                           torch_dtype=torch.float32)
model      = model.float()
pipeline   = ZeroShotClassificationPipeline(
    model, tokenizer, classification_type='multi-label', device='cpu')

test_df = pd.read_csv('/data/test.csv')
labels  = ['disaster', 'not disaster']
predictions = []

for i, row in test_df.iterrows():
    text    = str(row['text'])
    result  = pipeline(text, labels, threshold=0.1)
    scores  = {pred['label']: pred['score']
               for item in result for pred in item}
    disaster_score = scores.get('disaster', 0)
    # BUG: 0.3 too low - most tweets score above this
    prediction = 1 if disaster_score > 0.3 else 0
    predictions.append(prediction)
    if i % 100 == 0:
        wandb.log({'progress': i, 'disaster_score': disaster_score})

submission = pd.DataFrame({'id': test_df['id'], 'target': predictions})
submission.to_csv('/data/submission.csv', index=False)
print(submission['target'].value_counts())
# Output: 1      3263 (ALL disaster - WRONG)
run.finish()
```

□ run_model.py v5 — Kaggle v2 (label comparison)

Purpose: Compare disaster vs not-disaster scores directly

✓ **❌ FAILED** — all 3263 tweets predicted as not disaster

❌ BUG: 'not disaster' label always scores higher → all predictions = 0

```
# run_model.py — Version 5: Kaggle v2 label comparison (FAILED)

import torch
import wandb
import pandas as pd
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

run = wandb.init(project='mlops-assignment3', name='kaggle-disaster-tweets-v2')

tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model      = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                           torch_dtype=torch.float32)
model      = model.float()
pipeline   = ZeroShotClassificationPipeline(
    model, tokenizer, classification_type='multi-label', device='cpu')

test_df = pd.read_csv('/data/test.csv')
labels  = ['disaster', 'not disaster']
predictions = []

for i, row in test_df.iterrows():
    text      = str(row['text'])
    result     = pipeline(text, labels, threshold=0.1)
    scores     = {pred['label']: pred['score']
                  for item in result for pred in item}
    disaster_score      = scores.get('disaster', 0)
    not_disaster_score = scores.get('not disaster', 0)
    # BUG: not_disaster always wins — model biased
    prediction = 1 if disaster_score > not_disaster_score else 0
    predictions.append(prediction)
    if i % 100 == 0:
        wandb.log({'progress': i, 'disaster_score': disaster_score})

submission = pd.DataFrame({'id': test_df['id'], 'target': predictions})
submission.to_csv('/data/submission.csv', index=False)
print(submission['target'].value_counts())
# Output: 0      3263 (ALL not disaster — WRONG)
run.finish()
```

□ run_model.py v6 – FINAL (median threshold)

Purpose: Use median of all disaster scores as dynamic threshold

✓ ✓ SUCCESS — 1631 disaster, 1632 not disaster. Kaggle score: 0.48605

✓ KEY INSIGHT: Collect all scores first → compute median → use as threshold → balanced 50/50 predictions

```
# run_model.py – Version 6 FINAL: Median threshold (BEST SCORE: 0.48605)
# Docker: docker run --rm -v model -v /data -v ~/.netrc gliclass-model

import torch
import wandb
import pandas as pd
import statistics
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

# Initialise W&B
run = wandb.init(
    project='mlops-assignment3',
    name='kaggle-disaster-tweets-v3'
)

# Load model
print('Loading model...')
tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                     torch_dtype=torch.float32)
model = model.float()

pipeline = ZeroShotClassificationPipeline(
    model, tokenizer,
    classification_type='multi-label',
    device='cpu'
)

# Load competition data
test_df = pd.read_csv('/data/test.csv')
train_df = pd.read_csv('/data/train.csv')
print(f'Test: {len(test_df)} tweets')
print(f'Train distribution:\n{train_df["target"].value_counts()}')

# Step 1: Collect all disaster scores
labels = ['disaster', 'not disaster']
disaster_scores = []

for i, row in test_df.iterrows():
    text = str(row['text'])
    # Use very low threshold to capture all scores
    result = pipeline(text, labels, threshold=0.01)
    scores = {pred['label']: pred['score']
              for item in result for pred in item}
    disaster_score = scores.get('disaster', 0)
    not_disaster_score = scores.get('not disaster', 0)
    disaster_scores.append(disaster_score)
    if i % 100 == 0:
```

```

        print(f'    {i}/{len(test_df)}'
              f' | disaster={disaster_score:.3f}'
              f' | not_disaster={not_disaster_score:.3f}')
        wandb.log({'progress': i,
                  'disaster_score': disaster_score,
                  'not_disaster_score': not_disaster_score})

# Step 2: Use MEDIAN as dynamic threshold
# This adapts to model's score distribution → balanced predictions
median_score = statistics.median(disaster_scores)
print(f'\nMedian disaster score: {median_score:.4f}')
# Our value: 0.4502

# Step 3: Predict using median threshold
predictions = [1 if s > median_score else 0 for s in disaster_scores]

# Step 4: Save submission CSV
submission = pd.DataFrame({'id': test_df['id'], 'target': predictions})
submission.to_csv('/data/submission.csv', index=False)

print('\n=== Final Results ===')
print(submission['target'].value_counts())
# Output: 1 → 1631, 0 → 1632 (balanced!)

# Log final metrics to W&B
wandb.log({
    'median_threshold': median_score,
    'total_disaster': int(sum(predictions)),
    'total_not_disaster': int(len(predictions) - sum(predictions))
})
run.finish()
print('Done! Submission saved and results logged to W&B!')

```

□ run_model.py v7 – Better Labels (latest attempt)

Purpose: Use longer descriptive labels for better model understanding

✓ Submitted for improved score beyond 0.48605

```
# run_model.py – Version 7: Descriptive labels

import torch
import wandb
import pandas as pd
import statistics
from gliclass import GLiClassModel, ZeroShotClassificationPipeline
from transformers import AutoTokenizer

run = wandb.init(project='mlops-assignment3',
                 name='kaggle-disaster-tweets-v4-better-labels')

tokenizer = AutoTokenizer.from_pretrained('/models/gliclass-llama-1.3B')
model      = GLiClassModel.from_pretrained('/models/gliclass-llama-1.3B',
                                           torch_dtype=torch.float32)
model      = model.float()
pipeline   = ZeroShotClassificationPipeline(
    model, tokenizer, classification_type='multi-label', device='cpu')

test_df = pd.read_csv('/data/test.csv')

# Better labels – more descriptive context for the model
labels = [
    'natural disaster emergency crisis fire flood earthquake',
    'normal everyday tweet unrelated to disaster'
]

disaster_scores = []
for i, row in test_df.iterrows():
    text = str(row['text'])
    result = pipeline(text, labels, threshold=0.01)
    scores = {pred['label']: pred['score']
              for item in result for pred in item}
    disaster_scores.append(scores.get(labels[0], 0))
    if i % 100 == 0:
        wandb.log({'progress': i,
                   'disaster_score': disaster_scores[-1]})

median_score = statistics.median(disaster_scores)
predictions = [1 if s > median_score else 0 for s in disaster_scores]

submission = pd.DataFrame({'id': test_df['id'], 'target': predictions})
submission.to_csv('/data/submission.csv', index=False)
print(submission['target'].value_counts())
wandb.log({'median_threshold': median_score,
          'total_disaster': int(sum(predictions))})
run.finish()
```

Section 7: Errors & Fixes

Error	Cause	Fix
pip not found	venv not activated	<code>source ~/mlops_venv/bin/activate</code>
Operation not permitted (venv)	NTFS drive blocks Linux perms	Create venv in ~/ not on F: drive
No such file or directory	Space in 'ML Ops' folder name	Wrap path in quotes: <code>"/mnt/f/ML Ops/..."</code>
VS Code server error (socket)	Stale WSL socket	<code>rm -rf /run/user/1000/vscode-ipc-*</code> <code>&& pkill -f vscode-server</code>
401 Unauthorized HF	HF_TOKEN env var overrides login	<code>os.environ.pop('HF_TOKEN', None)</code>
huggingface-cli deprecated	Old CLI removed	Use: <code>hf auth login</code>
W&B key < 40 chars	Wrong key copied	Get full key from wandb.ai/authorize
BFloat16 vs Float32 error	Model weights dtype mismatch	Add: <code>model = model.float()</code>
docker: command not found	WSL integration off	Docker Desktop → Settings → WSL Integration → FedoraRemix
Docker credential error	Broken config.json	<code>rm ~/.docker/config.json && echo '{}' > ~/.docker/config.json</code>
Permission denied deleting hf_cache	Docker vhd files locked	Stop Docker Desktop, then delete from Windows PowerShell as Admin
All tweets = disaster (score 1.0)	threshold=0.3 too low	Use median threshold approach
All tweets = not disaster	not_disaster label scores higher	Use median — don't compare labels directly
Kaggle 403 Forbidden	Haven't accepted rules	Join competition at kaggle.com first
kaggle: command not found	venv not activated	<code>source ~/mlops_venv/bin/activate</code>

Section 8: Final Summary

Component	Status	Details
WSL Fedora Remix	✓Done	Windows WSL with Fedora Remix distro
VSCode + Jupyter	✓Done	WSL mode, Python (MLOps-A3) kernel
Virtual Environment	✓Done	~/mlops_venv — all packages installed
Hugging Face Login	✓Done	User: ausafahmad
Model Downloaded	✓Done	gliclass-llama-1.3B → /mnt/f/models/
Docker WSL Integration	✓Done	Docker Desktop → FedoraRemix enabled
Docker Image Built	✓Done	gliclass-model:latest
Model in Docker (basic)	✓Done	Zero-shot classification working
Flask Web UI	✓Done	http://localhost:8080
Kaggle Auth	✓Done	ausafaitj, ~/.kaggle/kaggle.json
Kaggle Dataset	✓Done	/mnt/f/datasets/disaster-tweets/
W&B Connected	✓Done	Project: mlops-assignment3
W&B Runs Logged	✓Done	5 runs with metrics and charts
Kaggle Submission v1	⚠0.42966	threshold=0.3 — all tweets = disaster
Kaggle Submission v3 BEST	✓0.48605	Median threshold — 1631/1632 split